EXPERIMENT-12A

26/9/26

IMPLEMENTATION OF ECHO CLIENT SERVER USING TCP/UDP SOCKETS

AIM:

To implement an echo client server application using TCP/UDP sockets and send a message from the client to the server and receive the same message back.

PROCEDURE:

TCP ECHO SERVER:

→ Start the TCP server.
→ Create a server socket and wait for the client.
→ Accept the client connection.
→ Read the message sent by the client.
→ Send the same message back to the client.
→ Display the echoed message.
→ Close the connection.

UDP ECHO SERVER:

→ Start the UDP server.
→ Create a datagram socket.
→ Receive the message from the packet.
→ Send the same message back to its client.
→ Repeat until bye is received.
→ Close the socket.

UDP:

→ Start the UDP client
→ Create a datagram socket.
→ Read a message from user.
→ Create a datagram packet & send it to server.
→ Receiver the echoed packet from server.
→ Display the echoed message.
→ Close the sockets.

TCP ECHO CLIENT:

→ Start the TCP client.
→ Create a socket & connect to the server.
→ Read a message from the user.
→ Send the message to the server.
→ Receive the echoed message from the server.
→ Display this echoed message.
→ Close the connections.

CODE:

TCP ECHO SERVER;

```
import java.io.*;
import java.net.^;
public class TCPECHOSERVER{
    public static void main(String[] args)
    extends IOException
    {
        serversocketserver = new serversocket(5000);
        System.out.println("TCP echo server started");
        socket socket = server.accept();
        Buffered Reader in = new Buffered Reader();
        print written out = new print writter(
                                socket.get.out.putStream
                                (), true);
        string message;
        while((message = in.read.line()));
```

OUTPUT:

TCP Echo:

Server:

TCP echo server started
Received : Hello
Received : Welcome
Received : Bye

Client:

Connected to TCP Echo server
Enter message : Hello
Server echo : Hello
Enter message : Welcome
Server : Echo : Welcome
Enter message : Bye
Server : Echo : Bye

```
System.out.println("Received message");
out.println("Echo" + message);
if(msg.equals("bye"))
    break;
}
socket.close();
server.close();
```

TCP ECHO CLIENT:

```
import java.io.*;
import java.net.*;
public class TCPEchoClient{
    public static void main(String[] args)
    {
        Socket socket = new socket("local host", 5000);
        PrintWriter out = new print Writter(
                            socket.getout.putstream
                            (1.true);
        System.out.println("connected to TCP
                            Echo Server");
        string message;
        while(true)
            {
                system.out.println("Enter message");
            }
        socket.close();
    }
}
```

UDP ECHO SERVER:

```
import java.net.*;
public class UDPEchoServer {
    public static void main(String[] args)
        {
            System.out.println("UDP Echo Server
                                Started");
            byte[] butter = new byte[1024];
            while(true)
                {
                    socket.receive(packet);
                    String msg = new String(packet.
                                getData());
                    String response = "Echo" + message;
                    byte[] data = response.getbytes();
                    socket.send(reply);
                }
            socket.close();
        }
}
```

UDP ECHO CLIENT:

```
import java.net.*;
import java.io.*;
public class UDPECHOCLIENT{
        public static void main(String[] args)
            {
                Inet address address = Inet Address(
                                "get By name("Local host"
                                );
                System.out.println("connected to app.
                                Echo server");
```

```
while(true)
  {
    System.out.println("Enter message");
    byte[] data = message.getbytes();
    socket send(packet);
    byte[] buffer = new buffer[1024];
    socket.receive(reply);
    if(message.equalsIgnorecase("bye")
      {
        break;
      }
    socket.close();
  }
}
```

RESULT:

Thus, the echo client-server application was successfully implemented using TCP & UDP sockets & the message sent by the socket were successfully echoed by the server.

OUTPUT:

Server:

UDP Echo Server Started

Received : Hello

Received : Welcome

Received : Bye

Client:

Connected to UDP echo Server

Enter message : Hello

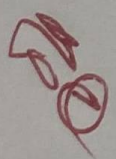
Server : Echo : Hello

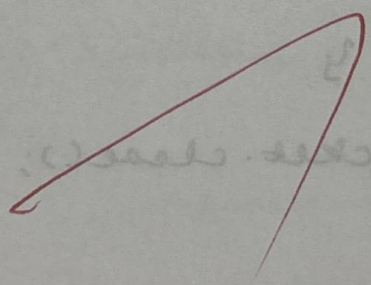
Enter message : ~~Hello~~ Welcome

Server : Echo : Welcome

Enter message : Bye

Enter Server : Echo : Bye





26/9/26

EXPERIMENT-12B

IMPLEMENT A CHAT PROGRAM USING SOCKET PROGRAMMING

AIM:

To implement a simple chat client server application using TCP sockets for communication between clients through a server.

ALGORITHM/PROCEDURE:

→ Start the TCP server & create a server socket.
→ Accept the client connection.
→ Create a thread to handle each client.
→ Read message sent by the client.
→ Broadcast the message to other connected clients.
→ Continue chatting until quit entered.
→ Close the connection.

PROGRAM:

```
Chat Server;

import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer
    {
     static Set <PrintWriter> clients =
     new HashSet(x);
     public static void main <String [] args>
     throw; Exception {

     Server socket server = new ServerSocket
                            (8000);

     while(true)
        {
```

```
sockets = server.accept();
new thread ()
    {
     try
      {
       BufferedReader in = new BufferedReader {
      new InputStreamReader in = new
      InputStreamReader (s.getInputStream()));
      s.getoutputStream().true;
      clients.add(ad);
      string msg;
      system.out.println(msg);
      }
      catch(Exception e)
       {
       }
   }).start();
  }
 }
}
```

Chart client:

```
import java.io.*;
import java.net.*;
public class chatclient
  {
    public static void main(string[] args) throws
    Exception
     {
      sockets s = new Socket("local host", 50000);
      print writer out = new printerwriter(s.getoutput
                                  stream(), true);
```

```
BufferedReader b = new BufferedReader(
new InputStreamReader(system));
new thread(c);
{
  try
  {
    string msg;
    while(cong = in.readline()) != null)
        {
          system.out.println(msg);
        }
    Catch(Exception e)
    {
    }).start();

  String msg;
  while((msg = = kb; readline()) != null)
  {
    out.println(msg)
  if(msg.equalIgnorecase("|quit") {
      break;
  }
  socket.close;
}
}
```

RESULT:

Thus, the chart client server application was successfully implemented using TCP sockets.

OUTPUT:

Server:

Chat server started
Alice joined the chat
Prashath joined the chat
Alice: Hello
Prashath: Hi

Client:

Enter your name:
Alice
Hello
Prashath: Hi

